CORE–2: DIGITAL LOGIC – Unit-2

Addition and Subtraction of Signed Numbers: All negative numbers are represented in memory by their 2's complement form. When two numbers are added, an overflow may occur if the number of bits in the resultant exceeds the number of bits for representation of the number. This happens when either both the numbers are positive or both are negative. **An overflow is said to be occurred if the carry into the sign-bit and the carry out of the sign-bit are different.**

Addition of two Signed Binary Numbers:

- When two signed binary numbers that are represented in 2's complement form are added, if the sign bit is 1 then the result is negative and to find the magnitude of the number the result is again complemented to its 2's complement form.
- If no overflow occurs and there is a carry out of the sign bit then the carry is discarded.

Example1: Add the two decimal numbers **+7 and +4** using 2's complement method.

Solution: The **2's complement** representations of +7 and +4 with 5 bits each are shown below.

$$\begin{array}{r} 1 \\ +7_{10} = 0\ 0\ 1\ 1\ 1_2 \\ +4_{10} = 0\ 0\ 1\ 0\ 0_2 \\ \hline = 0\ 1\ 0\ 1\ 1_2 \\ = 11_{10} \end{array}$$

The resultant sum contains 5 bits. So, there is no carry out from sign bit. The sign bit '0' indicates that the resultant sum is **positive**. So, the magnitude of sum is 11 in decimal number system. Therefore, addition of two positive numbers will give another positive number.

Example2: Add the two decimal numbers **-7** and **-4** using 2's complement method. Assume that each number is represented by 5 bits.

Solution: The **2's complement** representation of -7 and -4 with 5 bits each are shown below.

$$\begin{array}{r} 1\ 1 \\ -7_{10} = \ 1\ 1\ 0\ 0\ 1_2 \quad \text{(2's complement 7)} \\ -4_{10} = \ 1\ 1\ 1\ 0\ 0_2 \quad \text{(2's complement 4)} \\ \hline = 1\ 1\ 0\ 1\ 0\ 1_2 \\ = \ 1\ 0\ 1\ 0\ 1_2 \quad \text{(2's complement of } 1\ 0\ 1\ 0\ 1 \text{ is } 0\ 1\ 0\ 1\ 1 = 11_{10} \end{array}$$

$\therefore\ -7_{10} + (-4_{10}) = -11_{10}$

The resultant sum contains 6 bits. In this case, carry is obtained from sign bit. So, it is discarded. Resultant sum after removing carry is $\boldsymbol{10101_2}$. The sign bit '1' indicates that the resultant sum is **negative**. So, by taking 2's complement of it we will get the magnitude of resultant sum as 11 in decimal number system. Therefore, $-7_{10} + (-4_{10}) = -11_{10}$

Subtraction of two Signed Binary Numbers: Consider the two signed binary numbers A & B, which are represented in 2's complement form. We know that negative numbers are represented by their 2's complement form. So, whenever we have to subtract a number B from number A, then we take 2's complement of B and add it to A. So, **mathematically** we can write it as

A - B = A + *2's complement of B*

So, the subtraction of two signed binary numbers is similar to the addition of two signed binary numbers. But, we have to take 2's complement of the number, which is supposed to be subtracted. This is the **advantage** of 2's complement technique.

Example3: Subtract the decimal numbers **+4** from **+7**using 2's complement method. Assume that each number is represented by 5 bits.

Solution: $7_{10} - 4_{10} = 7_{10}$ + 2's complement of 4_{10}.

$$\begin{array}{rl} & 1\ 1\ 1 \\ +7_{10} = & 0\ 0\ 1\ 1\ 1_2 \\ -4_{10} = & 1\ 1\ 1\ 0\ 0_2 \quad \text{(2's complement 4)} \\ \hline & = 1\ 0\ 0\ 0\ 1\ 1_2 \\ & = 0\ 0\ 0\ 1\ 1_2 \\ & = 3_{10} \end{array}$$

Here, the carry into the sign bit and the carry out of the sign bit are same. So there is no overflow and the carry out of the sign bit is discarded. The resultant sum after removing the carry is $\boldsymbol{00011_2}$**.** The sign bit '0' indicates that the resultant sum is **positive** and its magnitude is 3 in decimal number system. $\therefore 7_{10} - 4_{10} = +3$.

Example4: Subtract the decimal number **+7** from **+4** using 2's complement method. Assume that each number is represented by 5 bits.

Solution: $4_{10} - 7_{10} = 4_{10}$ + 2's complement of 7_{10}.

$$\begin{array}{rl} +4_{10} = & 0\ 0\ 1\ 0\ 0_2 \\ -7_{10} = & 1\ 1\ 0\ 0\ 1_2 \quad \text{(2's complement 7)} \\ \hline & = 1\ 1\ 1\ 0\ 1_2 \\ & = -3_{10} \qquad \text{(2's complement of 1 1 0 1 is 0011} = 3_{10}) \end{array}$$

Here, carry is not obtained from sign bit. The sign bit '1' indicates that the resultant sum is **negative**. So, by taking 2's complement of it we will get the magnitude of resultant sum as 3 in decimal number system. Therefore, $4_{10} - 7_{10} = -3$

Example5: Add the two decimal numbers **+8 and +9** using 2's complement method. Assume that each number is represented by 5 bits.

Solution: The **2's complement** representations of +7 and +4 with 5 bits each are shown below.

$$\begin{array}{rl} & 1 \\ +8_{10} = & 0\ 1\ 0\ 0\ 0_2 \\ +9_{10} = & 0\ 1\ 0\ 0\ 1_2 \\ \hline & = 1\ 0\ 0\ 0\ 1_2 \end{array}$$

Here carry into the sign-bit is 1, but carry out of the sign-bit is 0 which indicates the **overflow condition**. The sign bit '1' indicates that the resultant sum is **negetive**. So, by taking 2's complement of it we will get the magnitude of resultant sum as 15

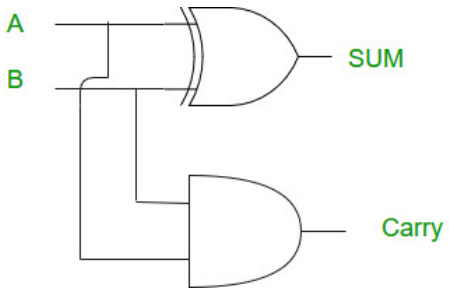
in decimal number system. **Therefore, $7_{10} + 9_{10} = -15_{10}$. The wrong result is produced because of overflow.**

Addition/ Subtraction Logic Unit:

Half Adder (HA): The addition of 2 bits is done using a combination circuit called half adder. The input variables are augends and addend bits and output variables are sum & carry bits.

Truth Table			
Inputs		Outputs	
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Logic Diagram



Logical Expression:

Sum = A⊕B

Carry = AB

Full Adder (FA): Full Adder is the adder which adds three inputs and produces two outputs. The first two inputs are A and B and the third input is an input carry as C_i. The output carry is designated as C_{i+1} and the normal output is designated as S which is SUM. A full adder logic is designed in such a manner that can take eight inputs together to create a byte-wide adder and cascade the carry bit from one adder to another.

Truth Table				
Inputs			Outputs	
A	B	C_i	Sum	C_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Logical Expression for SUM:

$= A'B'C_i + A'BC_i' + AB'C_i' + ABC_i$

$= C_i (A'B' + AB) + C_i'(A'B + AB')$

$= C_i (A \odot B) + C_i'(A \oplus B)$

$= A \oplus B \oplus C_i$

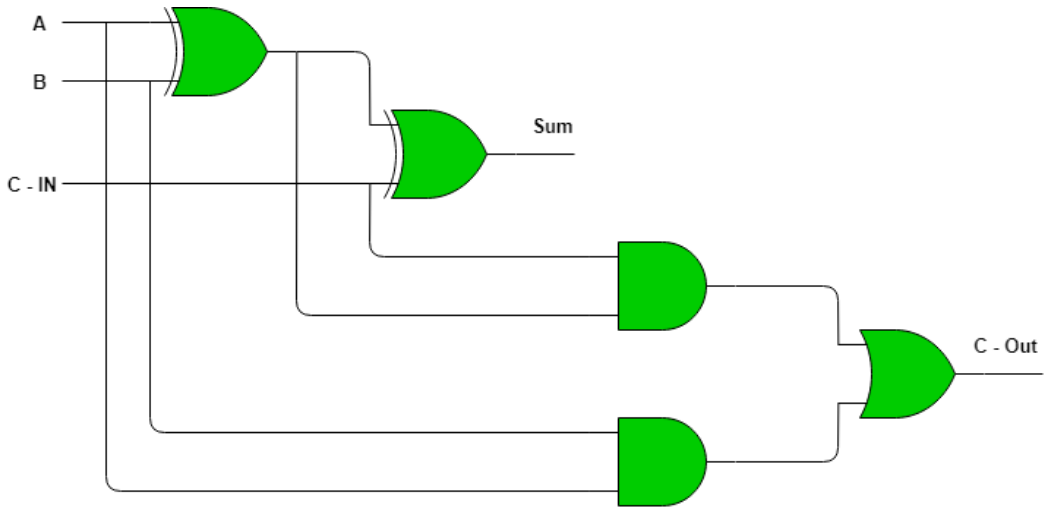
Logical Expression for C_{i+1}:

$= A' B C_i + A B' C_i + A B C_i' + A B C_i$

$= (A'B + AB') C_i + A B (C_i' + C_i)$

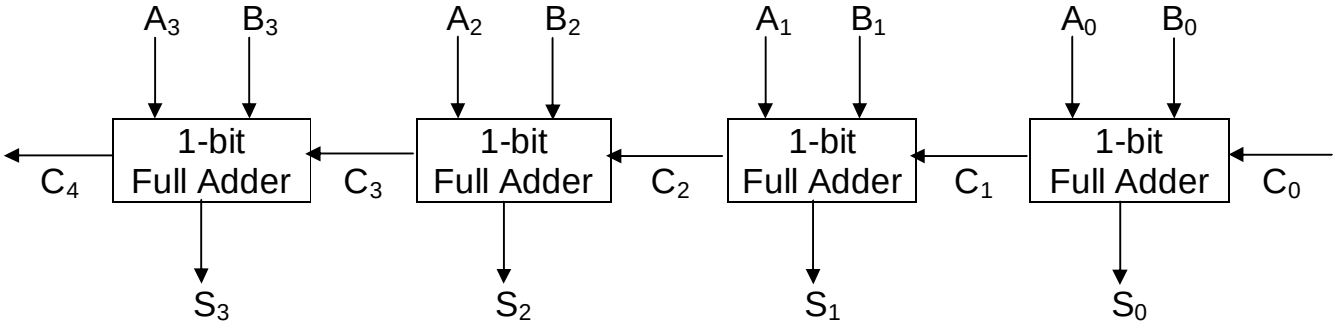
$= (A \oplus B)C_i + AB$

Logic Diagram of Full Adder:



Subtraction can be done by taking 2's complement of the subtrahend and adding it with the minuend. i.e. **A - B = A + *2's complement of B***

Ripple carry adders: In ripple carry adders, for each adder block, the two bits that are to be added are available instantly. However, each adder block waits for the carry to arrive from its previous block. So, it is not possible to generate the sum and carry of any block until the input carry is known. The ith block waits for the $(i-1)^{th}$ block to produce its carry. So there will be a considerable time delay which is carry propagation delay.



Consider the above 4-bit ripple carry adder. The carry input C_4 is not available on its final steady state value until carry C_3 is available at its steady state value. Similarly C_3 depends on C_2 and C_2 on C_1. Therefore, though the carry must propagate to all the stages in order that output S_3 and carry C_4 settle their final steady-state value. The propagation time is equal to the propagation delay of each adder block, multiplied by the number of adder blocks in the circuit. For example, if each full adder stage has a propagation delay of 20 nanoseconds, then S_3 will reach its final correct value after 60 (20 × 3) nanoseconds. The situation gets worse, if we extend the number of stages for adding more number of bits.

Carry Look-ahead Adder: A carry look-ahead adder reduces the propagation delay by introducing more complex hardware. In this design, the ripple carry design is suitably transformed such that the carry logic over fixed groups of bits of the adder is reduced to two-level logic. Let us discuss the design in details.

Consider the full adder circuit shown above with corresponding truth table. The logic expression for Sum S_i and Carry-out C_{i+1} of stage i are:

$$S_i = A_i \oplus B_i \oplus C_i \quad \text{and} \quad C_{i+1} = (A_i \oplus B_i)C_i + A_iB_i$$

We define two variables as **'carry generate'** G_i and **'carry propagate'** P_i , let

$$G_i = A_iB_i \quad \text{and} \quad P_i = A_i \oplus B_i$$

Now, The sum output and carry output can be expressed in terms of carry generate G_i and carry propagate P_i as

$$S_i = P_i \oplus C_i \quad \text{and} \quad C_{i+1} = G_i + P_iC_i$$

Where G_i produces the carry when both A_i, B_i are 1 regardless of the input carry. The carry output Boolean function of each stage in a 4 stage carry look-ahead adder can be expressed as

$$C_1 = G_0 + P_0 C_{in}$$
$$C_2 = G_1 + P_1 C_1 = G_1 + P_1 G_0 + P_1 P_0 C_{in}$$
$$C_3 = G_2 + P_2 C_2 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_{in}$$
$$C_4 = G_3 + P_3 C_3 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_{in}$$

From the above Boolean equations we can observe that C_4 does not have to wait for C_3 and C_2 to propagate but actually C_4 is propagated at the same time as C_3 and C_2. Since the Boolean expression for each carry output is the sum of products so these can be implemented with one level of AND gates followed by an OR gate.

Advantages and Disadvantages of Carry Look-Ahead Adder:
Advantages –

- The propagation delay is reduced.
- It provides the fastest addition logic.

Disadvantages –

- The Carry Look-ahead adder circuit gets complicated as the number of variables increase.
- The circuit is costlier as it involves more number of hardware.

Floating point representation and computer arithmetic: Computers use binary arithmetic, representing each number as a binary number. Computers use 2 formats for numbers. Fixed-point numbers are used to store integers that have a limited range. An alternative to fixed-point is floating-point numbers that approximate real numbers.

The IEEE Standard for Floating-Point Arithmetic (IEEE 754) is a technical standard for floating-point computation which was established in 1985 by the **Institute of Electrical and Electronics Engineers (IEEE)**. The standard addressed many problems found in the diverse floating point implementations that made them difficult to use reliably and reduced their portability. IEEE Standard 754 floating point is the most common representation today for real numbers on computers, including Intel-based PC's, Macs, and most UNIX platforms.

There are several ways to represent floating point number but IEEE 754 is the most efficient in most cases. IEEE 754 has 3 basic components:

1. **The Sign of Mantissa –** This is as simple as the name. 0 represents a positive number while 1 represents a negative number.
2. **The Biased exponent –** The exponent field needs to represent both positive and negative exponents. A bias is added to the actual exponent in order to get the stored exponent.
3. **The Normalised Mantissa –** The mantissa is part of a number in scientific notation or a floating-point number, consisting of its significant digits. Here we have only 2 digits, i.e. 0 and 1. So a normalised mantissa is one with only one 1 to the left of the decimal. **There are two ways of representing floating point numbers in IEEE 754 Standard: single precision and double precision.**

32 Bits

Sign	Exponent	Mantissa
1 Bit	**8 Bits**	**23 Bits**

(IEEE 754 Single Precesion Floating Point Representation)

64 Bits

Sign	Exponent	Mantissa
1 Bit	**11 Bits**	**52 Bits**

(IEEE 754 Double Precesion Floating Point Representation)

Example – Represent 85.125 in IEEE 754 Single Precision and Double Precision format.

85 = 1010101
0.125 = 001
85.125 = 1010101.001 = 1.010101001 x 2^6
sign = 0

1. Single precision:
biased exponent 127+6=133
133 = 10000101
Normalised mantisa = 010101001
we will add 0's to complete the 23 bits
The IEEE 754 Single precision is: 0 10000101 01010100100000000000000
This can be written in hexadecimal form as **42AA4000**

2. Double precision:
biased exponent 1023+6=1029
1029 = 10000000101
Normalised mantisa = 010101001
we will add 0's to complete the 52 bits

The IEEE 754 Double precision is:
= 0 10000000101 0101010010000000000000000000000000000000000000000000
This can be written in hexadecimal form as **4055480000000000**

Multiplying Floating Point Numbers

Algorithm to multiply two floating point numbers, x and y

1. Convert these numbers in scientific notation, so that we can explicitly represent hidden 1.
2. Let ‘a’ be the exponent of x and ‘b’ be the exponent of y.
3. Assume resulting exponent c = a+b. It can be adjusted after the next step.
4. Multiply mantissa of x to mantissa of y. Call this result m.
5. Normalize mantissa m and adjust exponent c to compensate.
6. XOR the two sign bits to get the resultant sign bit.

Example :- Multiply the two numbers: A = 1.11 x 2^0 and B = 1.01 x 2^2

1. Given, A = 1.11 x 2^0 and B = 1.01 x 2^2
2. So, exponent c = a + b = 0 + 2 = 2 is the resulting exponent.
3. Now, multiply 1.11 by 1.01, so result will be 10.0011
4. We need to normalize 10.0011 to 1.00011 and adjust exponent 1 by 3
5. Resulting sign bit 0 (XOR) 0 = 0, means positive.
6. So, the result after multiplication is 1.00011 x 2^3

Booth's Multiplication Algorithm:

Notations:
AC = Accumulator used to store the result

BR = B Register that stores the multiplicand

QR = Q Register that stores the multiplier

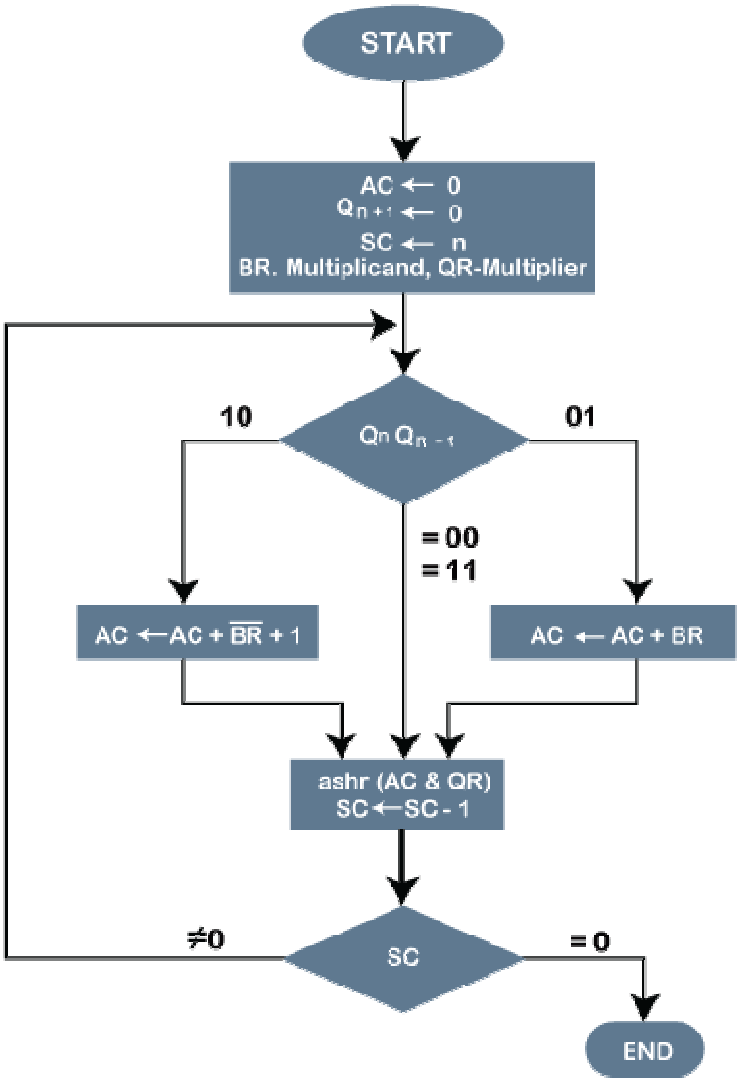
SC = Sequence Counter which initially stores the number of bits of the multiplier. It also indicates the number of iteration of the algorithm.

Q_n = Last bit (Least Significant Bit) of QR

Q_{n+1} = A bit which is initially set to 0

ashr = Arithmatic shift right

Here is the Flowchart of the Booth's Algorithm.



The algorithm works as follows:

1. If the two bits Q_nQ_{n+1} are equal to 10; it means that we have to subtract the multiplicand from the partial product in the accumulator AC and then perform the arithmetic shift operation, **ashr(AC,QR**) including $\mathbf{Q_{n+1}}$.
2. If the two bits Q_nQ_{n+1} are equal to 01, it means we need to perform the addition of the multiplicand to the partial product in accumulator AC and then perform the arithmetic shift operation, **ashr(AC,QR**) including $\mathbf{Q_{n+1}}$.
3. If the two bits Q_nQ_{n+1} are equal to 00 or 11, it means we need to perform only the arithmetic shift operation, **ashr(AC,QR**) including $\mathbf{Q_{n+1}}$.

The arithmetic shift operation is used in Booth's algorithm to shift AC and QR bits to the right by one and remains the sign bit in AC unchanged. In each iteration, the sequence counter is decremented by 1 until it is equal to zero. When SC becomes zero, the contents of AC & QR store the result of the multiplication.

Example-1: Multiply the two numbers 7 and 3 by using the Booth's multiplication algorithm.

Ans. We know $(7)_{10} = (0111)_2$ $(3)_{10} = (0011)_2$
Let multiplicand = 7 and multiplier = 3
So, BR = 0111 and QR = 0011
Set AC = 0000 and SC = 4 (as the number of bits here is 4)

as **BR = 0111** => BR′ = 1000 => **BR′+1 = 1001**

Iteration	Operation	AC	QR Q_n	Q_{n+1}	SC
	Initial values	0 0 0 0	0 0 1 **1**	0	4
1	AC = AC+ BR′+1	1 0 0 1	0 0 1 **1**		
	ashr(AC,QR), SC=SC-1	1 1 0 0	1 0 0 **1**	1	3
2	ashr(AC,QR), SC=SC-1	1 1 1 0	0 1 0 **0**	1	2
3	AC = AC+ BR	0 1 0 1	0 1 0 **0**		
	ashr(AC,QR), SC=SC-1	0 0 1 0	1 0 1 **0**	0	1
4	ashr(AC,QR), SC=SC-1	**0 0 0 1**	**0 1 0 1**	0	0

The numerical example of the Booth's Multiplication Algorithm is 7 x 3 = 21 and the binary representation of 21 is 10101. Here, we get the resultant in binary 00010101. Now we convert it into decimal, as $(000010101)_2$ = 2^4 + 2^2 + 2^0 = $(21)_{10}$.

Example-2: Multiply the two numbers 7 and 3 by using the Booth's multiplication algorithm.

Ans. We know $(7)_{10} = (0111)_2$ $(-3)_{10} = (1101)_2$
Let multiplicand = 7 and multiplier = -3
So, BR = 0111 and QR = 1101
Set AC = 0000 and SC = 4 (as the number of bits here is 4)
as **BR = 0111** => BR′ = 1000 => **BR′+1 = 1001**

Iteration	Operation	AC	QR Q_n	Q_{n+1}	SC
	Initial values	0 0 0 0	1 1 0 **1**	0	4
1	AC = AC+ BR′+1	1 0 0 1	1 1 0 **1**		
	ashr(AC,QR), SC=SC-1	1 1 0 0	1 1 1 **0**	1	3
2	AC = AC+ BR	0 0 1 1	1 1 1 **0**		
	ashr(AC,QR), SC=SC-1	0 0 0 1	1 1 1 **1**	0	2
3	AC = AC+ BR′+1	1 0 1 0	1 1 1 **1**		
	ashr(AC,QR), SC=SC-1	1 1 0 1	0 1 1 **1**	1	1
4	ashr(AC,QR), SC=SC-1	**1 1 1 0**	**1 0 1 1**	1	0

The numerical example of the Booth's Multiplication Algorithm is 7 x (-3) = -21 and the binary representation of 21 is 11101011. As the MSB is 1 so the result is **negative** and the magnitude is the 2's complement of 11101011 which is nothing but $(000010101)_2 = (21)_{10}$. So, $(11101011)_2 = (-21)_{10}$.